

Maintain version history here

VERSION 1.0.0

PROJECT PROPOSAL

VERSION HISTORY				
VERSION	APPROVED BY	REVISION DATE	DESCRIPTION OF CHANGE	AUTHOR
1.0.0				



Software Design Specification

for

Note Genius

Prepared by Group 35

Aleena Abbas

Muhammad Abubakar Minhas

Muhammad Hammad Yousaf

Muhammad Rayed

Muhammad Tayyab Haider

Date <09-03-25>

Effort <50 Hours>

Instructor

Naveed Arshad

Suleiman Shahid

SBASSE-LUMS

Contents

CONTENTS	III
1 INTRODUCTION	1
1.1 CHANGE LOG	1
1.2 DOCUMENT PURPOSE	1
1.3 Product Scope	
2 OVERALL DESCRIPTION	2
2.1 SYSTEM OVERVIEW	2
2.2 SYSTEM CONSTRAINTS	2
2.3 ARCHITECTURAL STRATEGIES	2
3 SYSTEM ARCHITECTURE	3
3.1 SYSTEM ARCHITECTURE	4
3.2 DATABASE MODEL	5
3.3 EXTERNAL INTERFACE REQUIREMENTS	6
4 USE CASE VIEW	6
4.1 USE CASE TABLE	6
4.2 USE CASE DIAGRAM	6
4.3 Use Case Description	
5 USER INTERFACE DESIGN	8
5.1 DESCRIPTION OF THE USER INTERFACE	8
5.2 DATA FLOW DIAGRAMS	8
5.3 SCREENS	8
5.4 USER INTERFACE DESIGN RULES	8
APPENDIX A - GROUP LOG	10
APPENDIX B – REFERENCES	11
A Definitions, Acronyms, and Abbreviations	
B References and Acknowledgements	

1 Introduction

1.1 Change log

No Changes.

1.2 Document Purpose

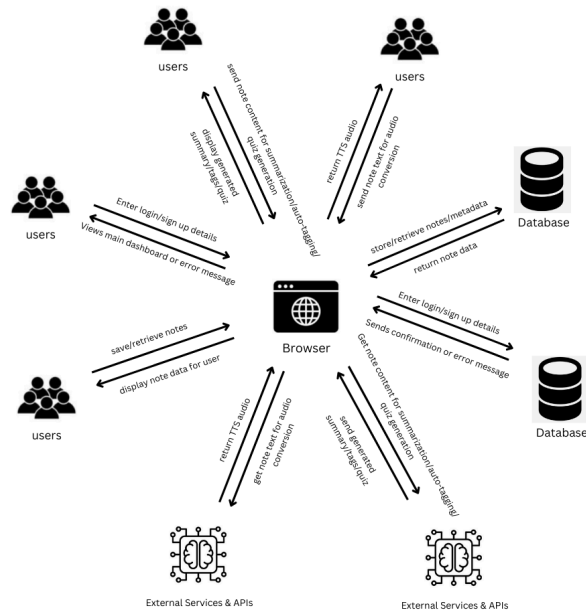
This Software Design Specification (SDS) document outlines the requirements and architecture for the AI-powered note-taking application. The product, currently in its initial development phase, is designed to assist users in creating, organizing, and enhancing notes through conventional and AI-driven features. The system leverages a large language model (LLM) for automated tagging, summarization, quick quiz creation, improving user experience and efficiency. This document specifies the core functionalities, system components, and interaction flows necessary for the development and implementation of the application.

1.3 Product Scope

The application is built as a standalone platform with a modular architecture, enabling features like auto-tagging, AI-generated summaries, and quiz generation to function cohesively within a unified interface. Its design emphasizes simplicity and practicality, targeting students and professionals who require structured note management.

2 Overall Description

2.1 System overview



This note taking app leverages the power of advanced AI capabilities to bring multiple features into one application enhancing the learning experience of our users (students, researchers, professionals etc.). The app allows users to not just to create/edit notes but also includes various functionalities such as AI prompted summarization, auto-tagging with AI, contextual assistance using AI, modifications suggested by AI to enhance notes, export to pdf, multimedia support and text to speech for visual/auditory learners, quiz generation along with search capabilities and authentication.

Users can register them on the app and their log in details are secured on our database, furthermore their data is isolated, meaning that other users can't access their notes. Apart from creating and editing notes that are also saved on our database, users can avail other features our app offers that make their learning experience far more efficient and effective. Navigating through notes is made easier with using AI auto tagging for organizing notes and users can search for their notes using relevant titles and tags etc. These notes can also be exported as PDF or markdown files for sharing or storing locally and to enhance the note taking app, multimedia support is also available. Additionally, AI powered features as mentioned above, play a crucial role in amplifying the user's note taking and learning experience. The platform also provides accessibility and allows users to multitask using a screen reader/text to speech feature.

Overall, this note taking application serves as a comprehensive platform that brings together AI powered functionalities and basic note taking all into one application.

2.2 System constraints

Hardware and Software Environment: The application would be mostly running on laptops and desktops, hence it would need cross-platform compatibility with multiple OSes (windows, macOS, Linux) and browsers (Chrome, Firefox, Edge, Safari) to work optimally

Performance Requirements: Since we are offering access to multiple AI driven tools and features and also going to handle a large number of concurrent users, solid error handling, optimized memory utilization and database query and API handling is crucial to maintain optimal performance with maximum uptime and software bugs prevention.

Availability of network and resources: The application is web based and thus will require a steady internet connection to be able to provide AI features and cloud-based storage reliant on external services which can have rate limits or downtimes.

Memory and other capacity limitations: The database should scale, and effective utilization of memory management should be achieved so that lots of data in the form of notes and multimedia files can be stored without any degradation of performance.

Security requirements: User information will be encrypted and limited to the user alone, so data is isolated and can't be manipulated by a different user. Security authentication features (email/password) will be applied to provide a secure working and learning environment for the user.

2.3 Architectural strategies

The architectural layers we are using for our application strike a compromise between performance and design requirements, scalability, flexibility, maintainability, and user experience. The strategies used are as followed:

Frontend in JavaScript (React.js): Since our note-taking app integrates dynamic AI features, a simpler, smoother UI is required to ensure a more responsive experience. The React.js framework single handedly divides the user interface (UI) into component based parts which are reusable, meaning the application will be responsive with an increased level of complexity within the UI while usability is not sacrificed.

Backend in Python (Flask): Flask is a lightweight framework that works excellent with API calls. As the user demand increases, the codebase is very easy to maintain and expand due to modularity and scalability. Our application is primarily served by functions through services like MongoDB and cloud based AI APIs, and Flask is able to flexibly connect these services enhancing and simplifying the user experience.

Database in NoSQL (MongoDB/PostgreSQL): The program will be processing multiple notes and accompanying metadata from several users, which as the user base increases, will become more and more complicated. MongoDB is well equipped with features that allow it to perform in-memory caching and indexing for frequently accessed data. This will make the application faster and more responsive and will also reduce the load on the database which aids in handling a growing number of concurrent users. The user can access and search through their notes more efficiently and this makes MongoDB a good strategy for our database. PyMongo or SQLAlchemy may be used to facilitate database interactions.

RESTful APIs: Use of RESTful APIs in our application ensures smooth communication between the multiple architectural layers of our application (frontend and backend), furthermore they allow for the integration of external services like the cloud-based AI APIs in our application. They make the application flexible, scalable and ensure a modular design which results in improved interoperability and the application can be used by any client (any browser) while maintaining quick access to data. This leads to improved performance and user experience. Additionally, RESTful APIs also leverage standard security practices to achieve a safe and secure environment.

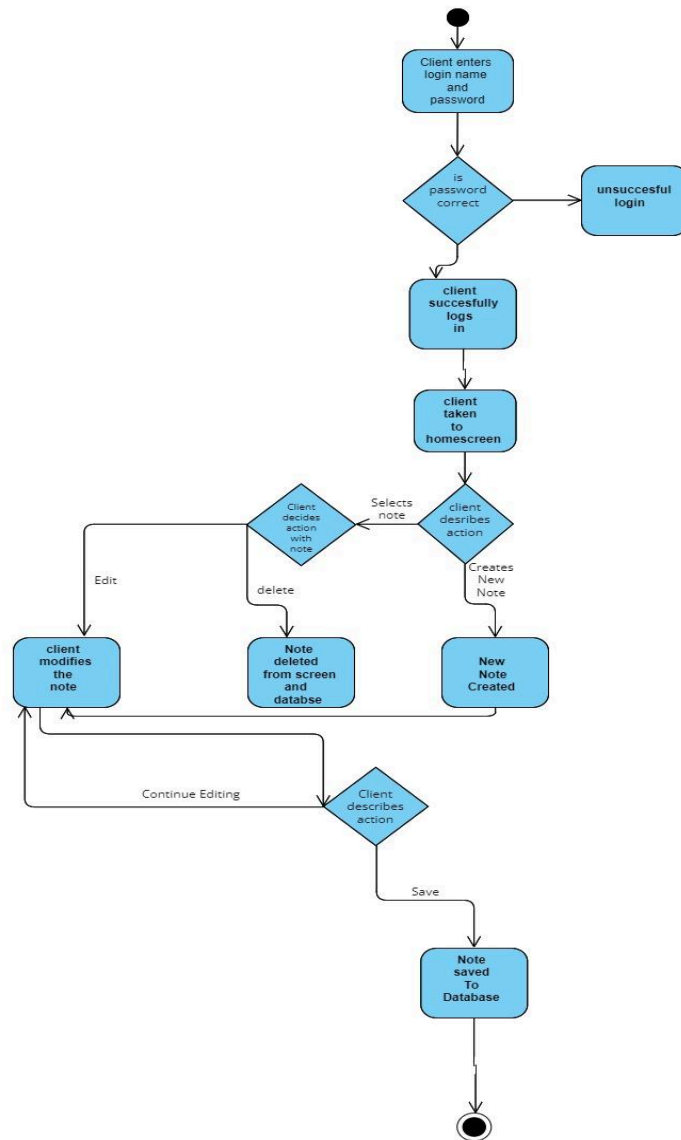
Existing external services: Our application provides the user with AI and other accessibility features to enhance their learning experience and to facilitate these features we need to use AI models and other cloud-based AI APIs. We make use of already existing AI and other external services such as pre-trained NLP models for AI powered features and external TTS APIs for our text-to-speech feature.

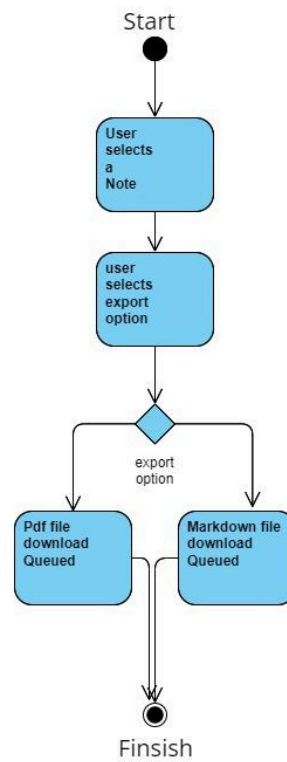
Ultimately, our goal is to organize a scalable, flexible, accessible, and maintainable system that adheres to the modern performance and design requirements that prioritize user experience. The architectural frameworks we have decided to use, satisfy these demands and aid in providing an effective and user friendly method to enhance learning and growth.

3 System Architecture

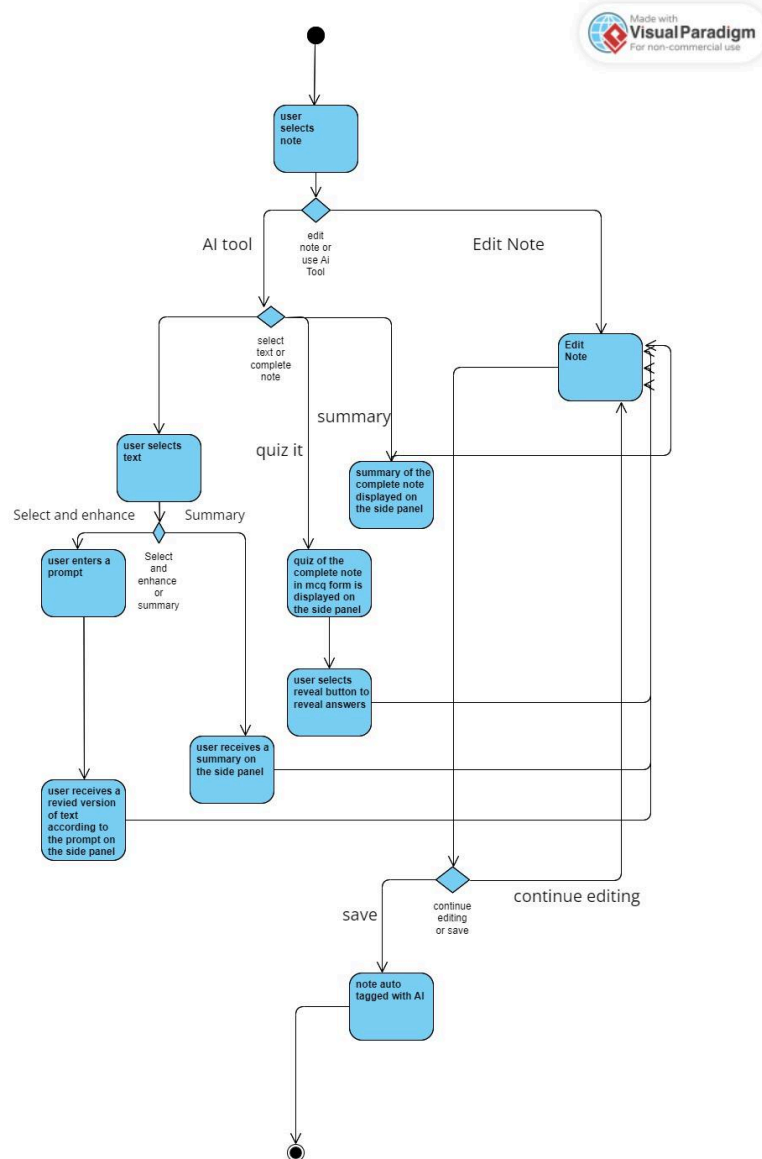
3.1 System Architecture

Use Case 1

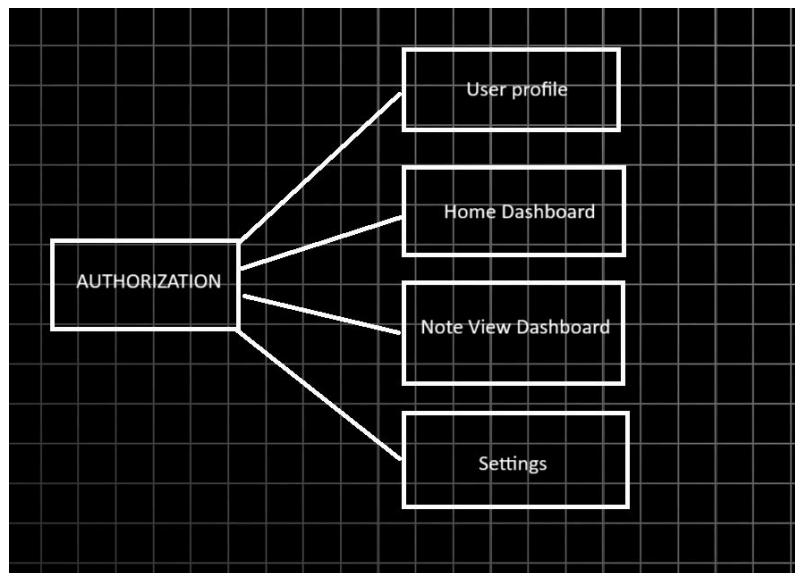


Use Case 2

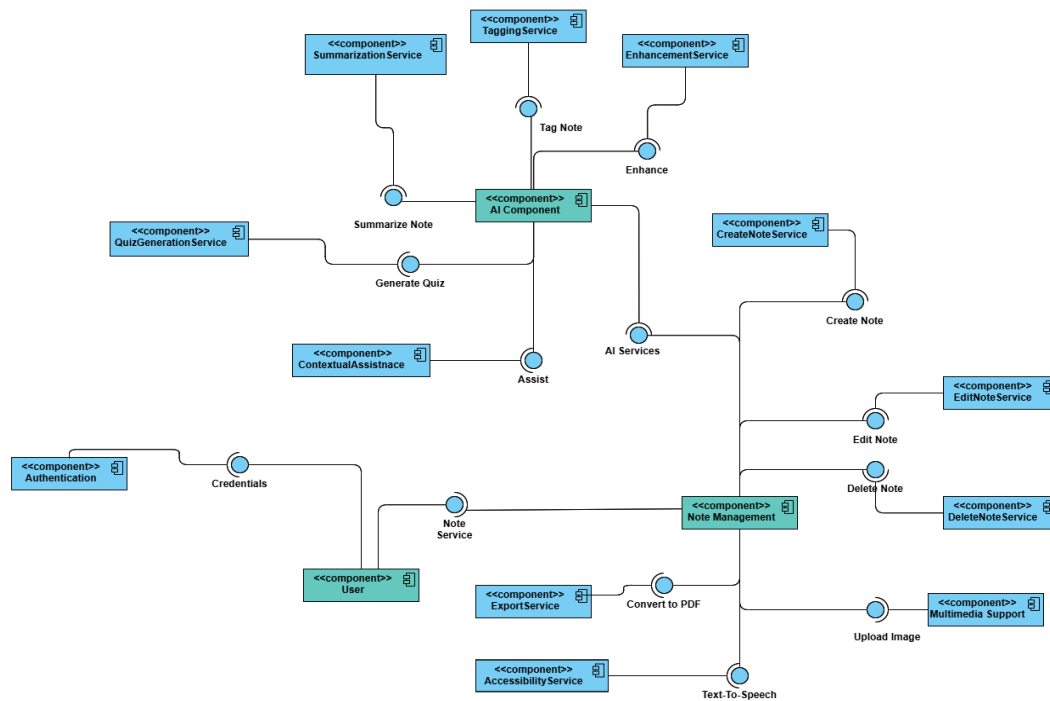
Use Case 3



Box-Line-Diagram



UML Component Diagram



3.2 Database Model

3.2.1 Database scheme and detailed description

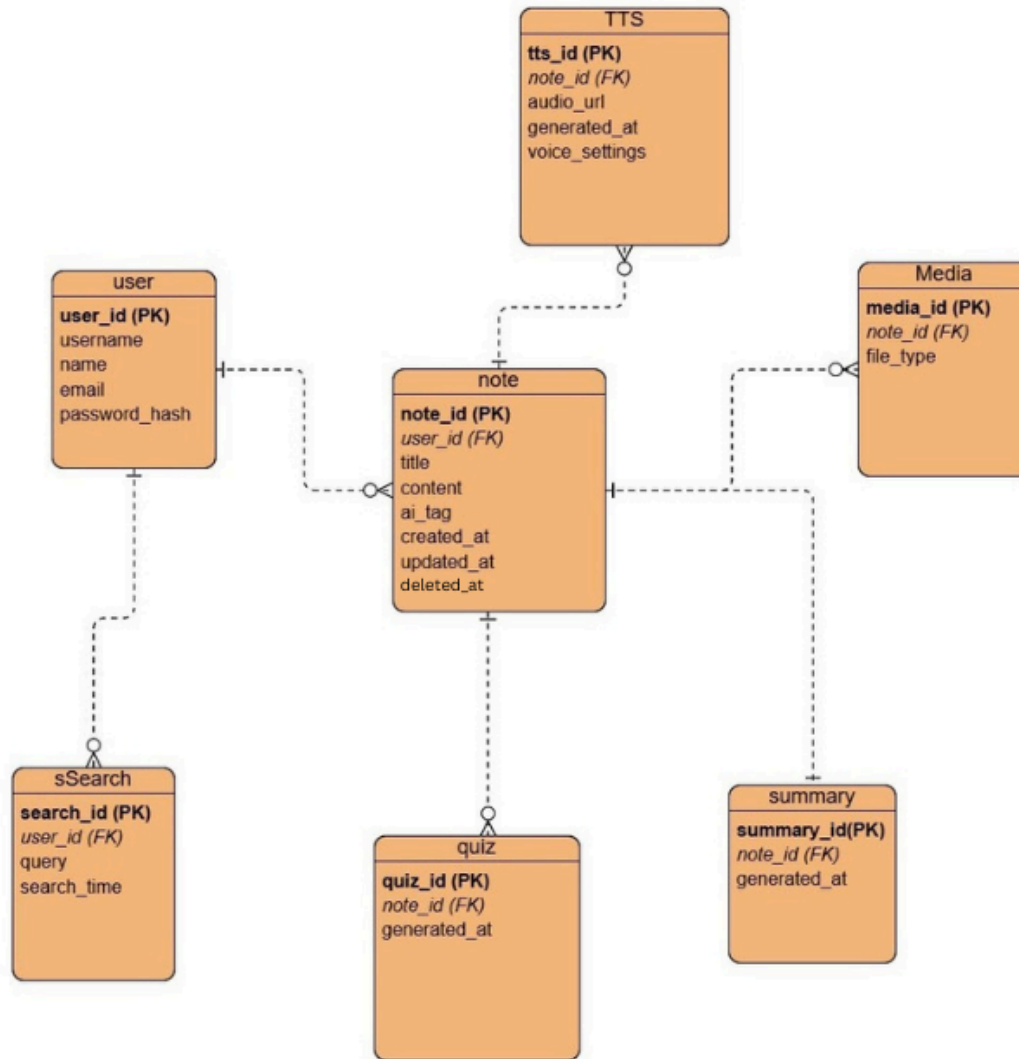


Table: user

Fields: user_id, username, email, password_hash, name

Aligns with Use Cases: Authentication , Login, View Personal Profile, Summary Generation, Quiz-It.

Purpose: Manages user information, authentication, and personalization

Table: note

Fields: note_id, user_id, email, title, content, ai_tag, created_at, updated_at, deleted_at

Aligns with Use Cases: Create Note, Edit Note, View Notes, AI Auto-Tagging, Search & Retrieval, Contextual Assistance, Enhancement

Purpose: Stores user-created notes, tracks modifications, and enhances searchability through AI-generated tags.

Table: search

Fields: search_id, user_id, query, search_time

Aligns with Use Cases: Search Notes, View Search History

Purpose: Stores user search queries to improve search functionality and provide recent search suggestions.

Table: TTS

Fields: tts_id, note_id, audio_url, generated_at, voice_settings

Aligns with Use Cases: Text-to-Speech, Accessibility Features, Audio Playback

Purpose: Stores generated speech files for notes, reducing API calls and improving playback performance.

Table: media

Fields: media_id, note_id, file_type

Aligns with Use Cases: Attach Media to Notes, View Multimedia Content

Purpose: Manages image attachments for notes, ensuring proper storage and retrieval.

Table: summary

Fields: summary_id, note_id, generated_at

Aligns with Use Cases: Summary Generation, AI-Assisted Study

Purpose: Stores AI-generated summary for notes, allowing users to review key points efficiently.

Table: quiz

Fields: quiz_id, note_id, generated_at

Aligns with Use Cases: Quiz System, Self-Assessment, AI-Powered Learning

Purpose: Stores quizzes generated from notes, enabling interactive learning and self-testing.

3.2.2 Database

Normalization and Data Integrity:

- The schema is **normalized** to minimize redundancy and ensure data consistency. Instead of storing user information in multiple places, the **user** table manages personal details, while related tables (**note**, **search**, **quiz**, **summary**, etc.) reference it using **foreign keys**.
- Referential integrity is enforced through **primary keys** (e.g., user_id, note_id, search_id) and **foreign keys** (e.g., user_id in the **note** table), ensuring that relationships between entities remain consistent.
- Keeping **summary** and **quiz** separate from **note** prevents **data duplication** while allowing a summary and multiple AI-generated quizzes to be linked to a single note. This ensures efficient storage and retrieval.

Scalability and Flexibility:

- The schema is structured to handle an increasing number of users, notes, and interactions efficiently. As more users generate notes, summaries, quizzes, and search queries, the database remains manageable without performance degradation.
- AI-based auto-tagging (stored in the `ai_tag` field of the **note** table) allows intelligent organization of notes without requiring a separate **tag** table, optimizing storage and retrieval.

Security Considerations:

- User authentication is handled via secure **password hashing** in the **user** table. The schema ensures that only authenticated users can access and modify their notes.
- The **search** table is user-specific, preventing unauthorized access to another user's search history.

Enhanced User Experience:

- The **search** table improves usability by storing recent searches, allowing users to quickly revisit past queries.
- The **TTS** table enhances accessibility by enabling users to listen to notes without making repeated API calls.
- Storing **AI-generated summaries** separately enables users to quickly review key points without manually summarizing content.
- The **quiz** table allows users to test their knowledge based on their notes, making the note-taking system more interactive.

Comprehensive Data Modeling:

- The schema captures all required functionalities of the **note-taking** app, including **note storage, AI-powered features (summaries, tagging, quizzes), search history, and media management**.
- The **media** table ensures that images are properly associated with notes.
- By keeping generated **summaries and quizzes** in separate tables, the system allows multiple versions of AI-generated content for the same note without modifying the original note.

3.3 External Interface Requirements

3.3.1 User Interfaces

The system will feature a **Graphical User Interface (GUI)** designed for desktop browsers, providing an intuitive and responsive experience.

Logical Characteristics of the Interface:

- **Dashboard:** Displays recent notes, search history, and quick actions.

- **Note Editor:** Rich-text editing with options for AI-generated summaries, quizzes, and text-to-speech playback.
- **Search Bar:** Supports full-text search and AI-generated tags for note retrieval.
- **Media Upload:** Allows users to attach images to notes.
- **Settings Page:** Personalization options, including voice settings for text-to-speech.
- **Export Functionality:** Enables exporting notes to PDF/Markdown.

Optimising the Interface:

- **Keyboard shortcuts** for quick navigation.
- **Text-to-Speech (TTS) integration** for users with reading disabilities.
- **Intuitive UI:** Clean layout with clear buttons for creating, editing, and organizing notes.
- **Dark Mode:** Improves accessibility for visually impaired users.

3.3.2 Hardware Interfaces

Web Interface (Client-Side):

- **Supported Devices:** Desktop computers, laptops with modern web browsers (e.g., Chrome, Firefox, Safari).
- **Data Interactions:** The web interface allows users to create, edit, and organize notes, perform searches, generate summaries, and export content. Data is exchanged between the frontend and backend via HTTP requests.
- **Control Interactions:** Users interact with the system through graphical user interface (GUI) elements such as text input fields, buttons, and menus. JavaScript libraries (e.g., React.js) handle dynamic content rendering and UI responsiveness.

Backend Server Interface (Flask/Python):

- **Supported Devices:** Server machines hosting the Flask-based backend.
- **Data Interactions:** The backend processes HTTP requests, handles authentication, and manages business logic. It communicates with the database to store and retrieve notes, search queries, and AI-generated content.
- **Control Interactions:** The Flask server routes API requests, processes user actions, and ensures secure data transactions. Background tasks like AI-generated summaries or text-to-speech processing are handled asynchronously.

Database Interface :

- **Supported Devices:** Servers hosting the PostgreSQL or MongoDB database.
- **Data Interactions:** The database stores structured data related to users, notes, searches, quizzes, summaries, and exported files. CRUD (Create, Read, Update, Delete) operations are performed by the backend to manage user-generated content efficiently.
- **Control Interactions:** The backend manages database connections, executes queries, and ensures data consistency. SQLAlchemy (for PostgreSQL) or PyMongo (for MongoDB) may be used to facilitate database interactions.

Text-to-Speech (TTS) Interface:

- **Supported Devices:** Desktop devices with speakers
- **Data Interactions:** The application may send text data to an external API to generate speech and retrieve an audio file. Alternatively, locally generated speech files may be stored in the database to reduce API calls.
- **Control Interactions:** Users can listen to notes by clicking the TTS option. The system allows customization of voice settings (e.g. speed).

Media Upload & Export Interface:

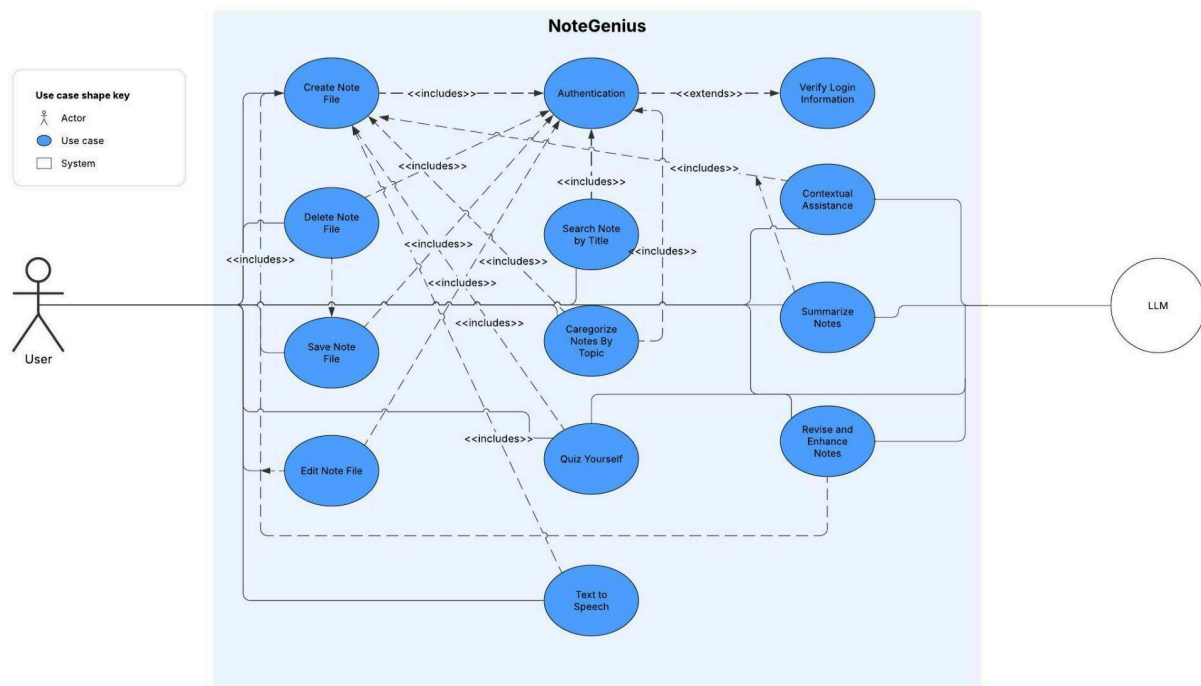
- **Supported Devices:** Desktop devices with local file storage capabilities
- **Data Interactions:** Users can upload media (images) to attach to notes. Exporting allows saving notes in different formats (e.g., PDF, Markdown).
- **Control Interactions:** Users can initiate media uploads through media selection. The backend validates, processes, and stores the media, ensuring compatibility with the note-taking system.

4 Use Case View

4.1 Use Case Table

Primary Actors	Associated Use Cases
User	Note File Creation
User	Note File Deletion
User	Note File Editing
User	Note File Saving
User	Adding an Image to a Note
User	Text to Speech
User	Search for Note Files
User	Contextual Assistance
User	AI Generated Summaries
User	Quiz it

4.2 Use Case Diagram



4.3 Use Case Description

Use Case 1: Note File Creation

Actors:

- User

Pre-Req:

- The user is logged in

Basic Flow:

1. Create a new note by clicking on "+ New Note" in the navbar
2. The note will not be created until the user names it uniquely
3. The user has to add a title to this note file
4. Start with a blank note file by default

Post-conditions:

- This note will then be added to the database
- This file can then be searched by title

Use Case 2: Note file Deletion

Pre-req:

- The user is logged in
- The user has at least one created Note file

Basic Flow:

1. The user presses the 3 dots in the toolbar to delete the opened note/ The user presses the 3 dots in the sidebar to delete other notes
2. The user can then click delete
3. A pop up will appear asking the user if they are sure they want to delete the file.

Post Condition:

- This note will be deleted from the database and will not appear in the sidebar displaying the notes as well.

Use Case 3: Note File Editing

Actors:

- User

Pre-Req:

- The user is logged in
- The note file exists

Basic Flow:

1. The user opens an existing note file
2. The user can edit the note using a multitude of features
3. The user manually saves the changes by clicking the Save Note button

Post-conditions:

- The modified note is updated in the database
- The updated note is accessible in its edited form

Use Case 4: Note File Saving

Actors:

- User

Pre-Req:

- The user is logged in
- A note file is open and has unsaved changes

Basic Flow:

1. The note file is saved manually by clicking the Save Note button.
2. The user is prompted to save the file if they attempt to close the note file

Post-conditions:

- The note file's latest version is stored in the database
- The saved note is available for future access and retrieval

Use Case 5: Adding an Image to a Note

Actors:

- User

Pre-Req:

- The user is logged in
- A note file is open

Basic Flow:

1. The user clicks on the image insert option in the toolbar
2. The user selects an image from their device
3. The image is added to the note in the chosen location(where the cursor is before importing the image)

Post-conditions:

- The image is embedded in the note file and saved in the database
- The note retains the image when reopened or shared

Use Case 6: Text-to-Speech

Actors:

- User

Pre-Req:

- The user is logged in
- A note file is open

Basic Flow:

1. The user selects a portion of text or enables text-to-speech for the entire note
2. The relevant API call is made and the text is converted into audio and played
3. The user can pause, stop, or adjust playback settings

Post-conditions:

- The note remains unchanged but has an audio playback option

Use Case 7: Search for Note Files

Actors:

- User

Pre-Req:

- The user is logged in
- At least one note exists in the database

Basic Flow:

1. The user enters a title to search for
2. The system retrieves and displays matching notes, with the one matching the most displayed on top
3. The user selects a note to open and view it

Post-conditions:

- The user can quickly find and access the desired note

Use Case 8 : Contextual Assistance

Actors:

- User

Pre-req:

- The user is logged in
- The user has a note file opened on the screen and it has some content/title written

Basic Flow:

1. The user selects the texts and clicks on the contextual assistance button
2. The app displays the related topics and info on a separate side panel.

Post Conditions:

- The user now has looked up more about the topic without having to switch to another app or screen
- If the user is willing, they can add the provided information to their note

Use Case 9: AI generated summaries

Actors:

- User

Pre-req:

- The user is logged in
- The user has a note file opened on the screen and it has some content written

Basic Flow:

1. The user highlights the relevant text and a dropdown appears
2. They then click “summarize” to generate a summary of the highlighted text in a separate window
3. The user clicks on the “Summarize Note” button which then generates a summary of the entire note in a separate window

Post Conditions:

- The user now has a summary of the note file in a separate window
- This summary, if generated from the entire note file, can then be saved
- It can be accessed in the toolbar

Use Case 10: Quiz it

Actors:

- User

Pre-req:

- The user is logged in
- The user has a note file opened on the screen and it has some content written

Basic Flow:

1. The user clicks the Quiz it button in the toolbar
2. A quiz along with it answers is generated in a separate window

Post Conditions:

- The user has now tested themselves on their notes without having to open/switch to another application.
- The user has also been saved the trouble of finding quizzing material elsewhere.
- The user is better equipped with knowledge of the relevant note file.
- The user can save the quiz and refer to it later.

5 User Interface Design

5.1 Description of the user interface

The user interface (UI) of our system is designed to be intuitive, responsive, and user-friendly. It consists of visually appealing components that ensure seamless user interaction and accessibility. The primary goals of the UI are to enhance user experience, provide easy navigation, and support multiple devices efficiently.

Front-End Tools and Technologies

We have chosen the following front-end technologies for developing the UI:

- **HTML5 & CSS3** – For structuring and styling the interface, ensuring a modern and responsive design.
- **React.js** – React.js is used for building dynamic and interactive UI components. It offers a component-based architecture, allowing reusability and modular development.

Rationale for Choosing These Technologies

- **Performance & Speed** – React.js allows virtual DOM usage, optimizing rendering speed.
- **Scalability** – A component-based UI ensures maintainability and easy scalability.
- **Cross-Platform Compatibility** – HTML, CSS, and JavaScript ensure the UI works well on both mobile and desktop devices.

Presentation Layer in MVC Architecture

In the **Model-View-Controller (MVC) architecture**, the **Presentation Layer** corresponds to the **View** component. This layer is responsible for displaying data to users and capturing user interactions. The UI in our system follows a structured MVC approach where:

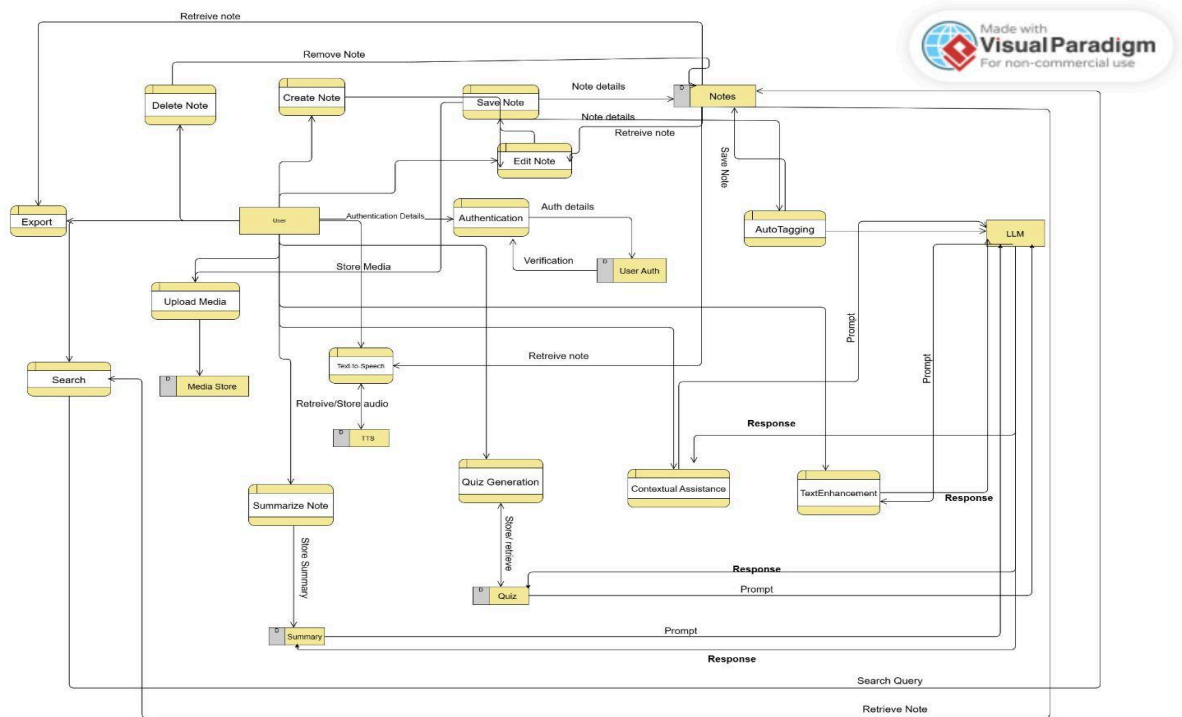
- The **View (Presentation Layer)** renders the user interface, including form inputs, buttons, and dynamic components.
- The **Controller** handles user interactions, processes requests, and updates the view accordingly.
- The **Model** maintains data integrity and interacts with the backend services.

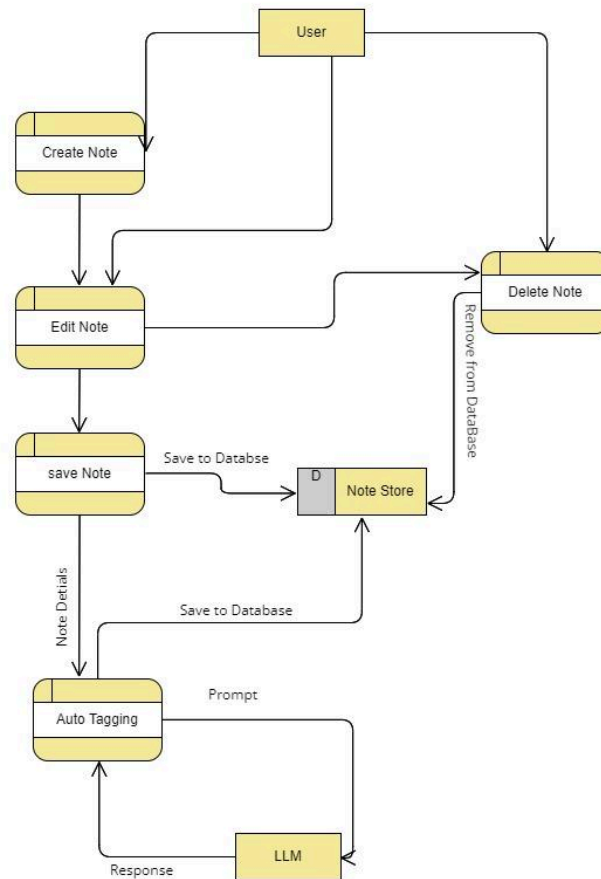
The UI ensures efficient rendering by using state management techniques, such as React's **useState** and **useEffect**, allowing dynamic updates without unnecessary re-renders. The Application also has

multiple Tabs such as the Pop up windows like Quiz-It and Summarize Functionalities which go very smoothly with the UI, using smooth animation Libraries.

5.2 Data Flow Diagrams

Level 0 (Context Diagram)





5.3 Figma Screens (UI)

Relationship Between Screens Progression of User Engagement: The screens are designed to guide users through a structured journey, from being a new visitor to becoming a fully registered and engaged user. After logging in, users are directed to the dashboard, where they can access their notes, settings, and other functionalities.

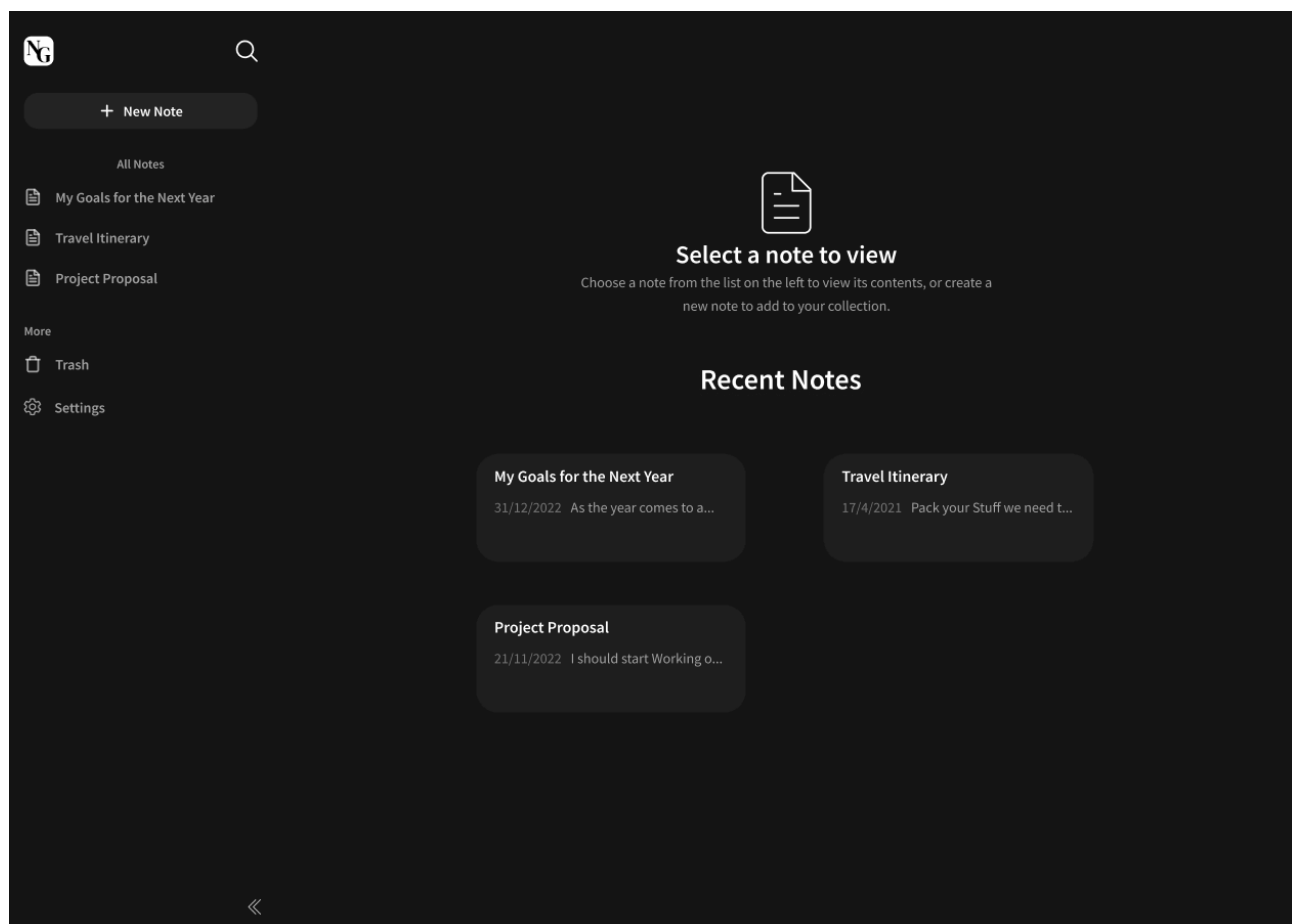
Consistency Across Screens: A left navigation panel remains consistent throughout the application for ease of access. This panel can be collapsed when the user enters a note (Zen Mode) to minimize distractions.

Course Overview Page

Screen Maps to the Use Case: View all the Current Notes, Recent Notes, Settings and a General Dashboard.

First Image: Dashboard Page where the user is sent after they Login to their account, Includes:

- **Main Dashboard:** Displays a prompt for users to select a note to start working on.
- **Recent Notes Shortcuts:** Allows users to quickly access their last edited notes.
- **Navigation Panel:**
 - "All Notes" tab: Displays all previously saved notes.
 - Clicking on a note navigates the user to the Note Page (Screen 2).
 - "More" tab: Includes options such as Trash (discarded notes) and Settings (opens Settings Page - Screen 3).
 - **Search Icon:** Clicking this reveals a search bar for searching notes.
 - **New Note Button:** Creates a new note, directing the user to Screen 2.
 - **Double Arrow Icon:** Collapses the left navigation panel (Zen Mode) for distraction-free note-taking.
 - **Logo (Top Left Corner):** Remains visible across all screens unless the navigation panel is collapsed.

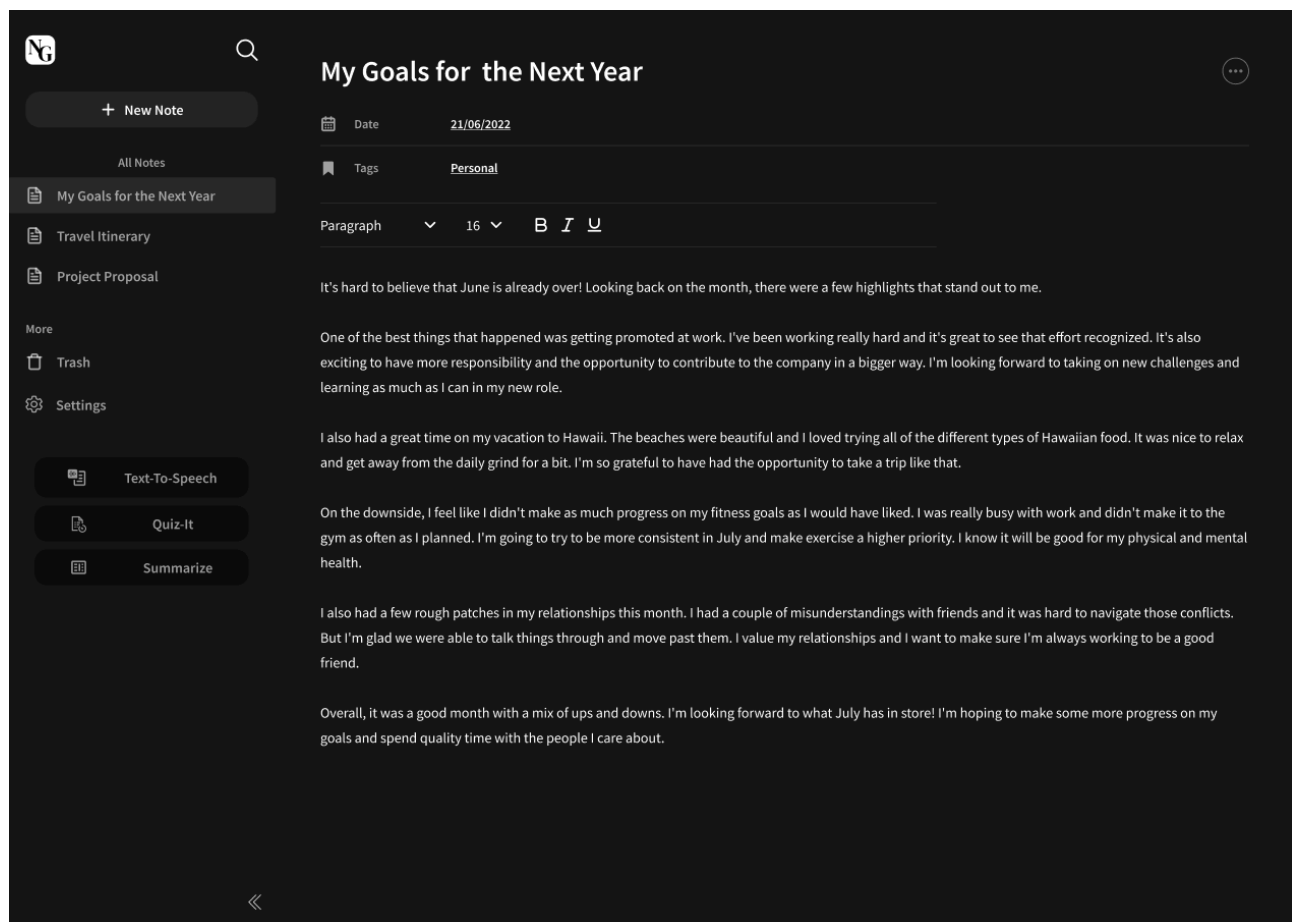


Course Overview Page

Screen Maps to the Use Case: View the Current Open Note and ability to manipulate or generate Content like Quiz Generation or Summarize text.

Second Image: Note Page which shows the Content of that Specific Note, Includes:

- **Left Navigation Panel:** Provides access to other notes and features, similar to the Dashboard Page.
- **Text-to-Speech Button:** Reads the note aloud, converting the text into an audio format.
- **Quiz-It Button:** Opens a right-side tab that generates a quiz based on the text content of the note.
- **Summarize Button:** Opens a right-side tab that provides an AI-generated summary of the note.
- **Note Header:** Displays the note title, date of creation, and auto-generated tags based on content analysis.
- **Text Formatting Tools:** Allows users to bold, italicize, underline, and format text (H1, H2, H3, etc.).
- **Note Settings Button:** Provides interactions such as renaming, deleting, or sharing the note.

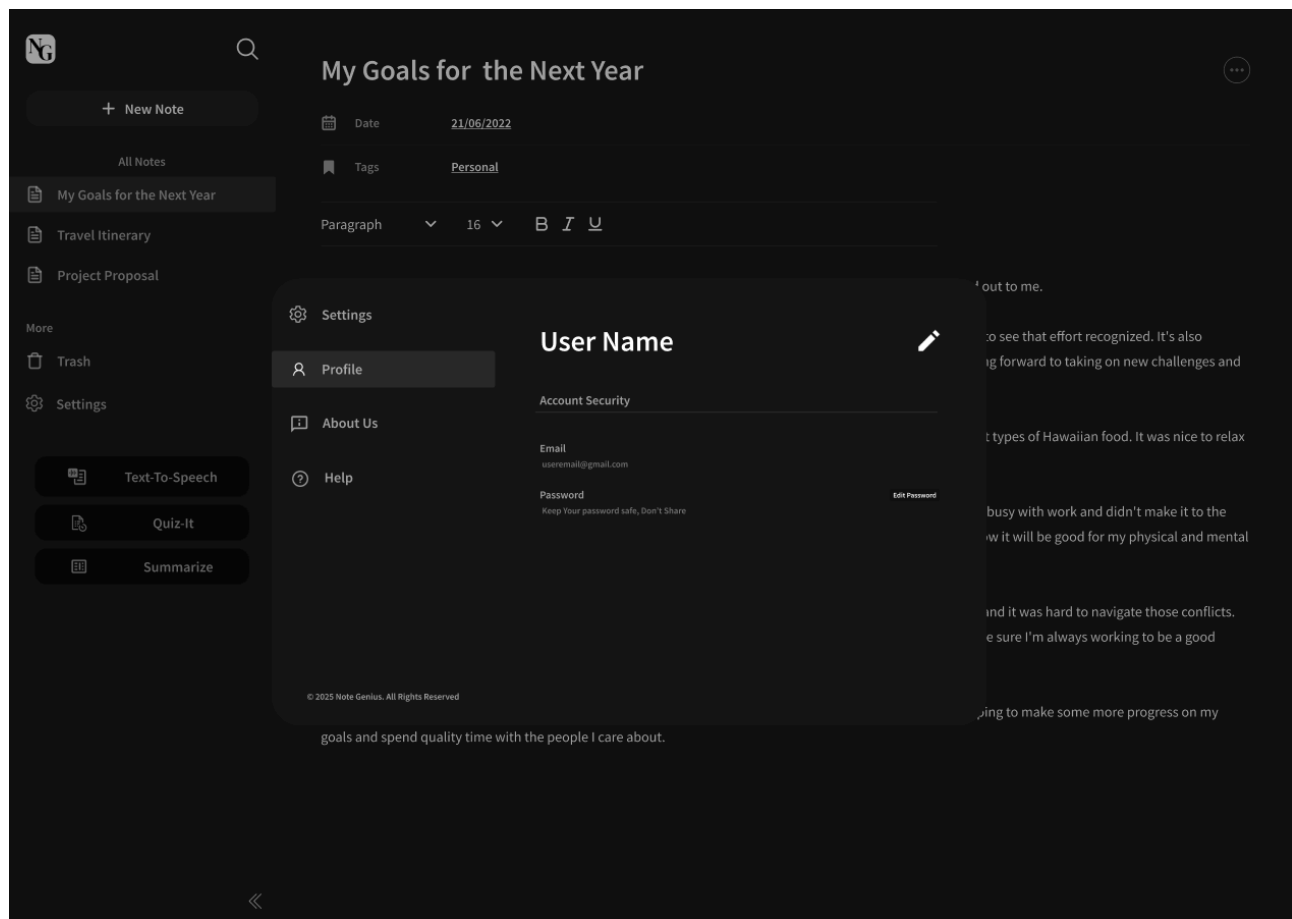


Course Overview Page

Screen Maps to the Use Case: Modify user preferences and access account-related functionalities.

Third Image: Setting Page Tab shown which has multiple tabs inside it, Including:

- **Settings Tab:**
 - Toggle between Dark Mode and Light Mode.
 - Adjust application-specific preferences.
- **Profile Tab:**
 - Displays user information such as name, email, and password.
 - Allows users to update their username and password using buttons next to the respective fields.
- **About Us Tab:**
 - Provides details about the team and contact information.
- **Help Tab:**
 - Includes user guides and support resources for better app usability.



5.4 User interface design rules

Appendix A - Group Log

Name	Contributions in this phase	Approx. Number of hours	Remarks
Muhammad Hammad Yousaf	Scrum Master, Section 1 (Introduction), Section 3 (System Architecture), Section 5 (Data flow Diagrams)	10	Good job
Muhammad Rayed	Section 4 (Use Case View)	10	Good job
Aleena Abbas	Section 3 (System Architecture), Section 5 (Data flow Diagrams)	10	Good job
M. Tayyab Haider	Section 2 (Overall Description)	10	Good job
Abubakar Minhas	Section 5.3, 5.1 (Figma Screens. Data Flow Diagrams, User Interface Design)	10	Good job

Appendix B – References

A Definitions, Acronyms and Abbreviations

LLM	Large Language Model
Backend Development	Server-Side Logic
Frontend Development	Client-Side Logic
RESTful APIs	A type of application programming interface that allows two computer systems to exchange information
UML	Unified Modelling Language
PyMongo	Library for MongoDB manipulation in python
NoSql	Not Only SQL, It refers to a category of databases

B References and Acknowledgment

Marking Rubric

Total marks: 45

System Constraints	3
System Architecture + UML	4
DB Model	5
Interface Reqs	3
Use Case View	10
Data Flow Diagrams	7
Figma Screens and UI Rules	8
References and Abbreviations	1
Concise and to-the-point descriptions	2
Writing Quality (Descriptions should be coherent and should covers all questions asked under the heading/follows format of document)	2